Wilson Ricardo Figueroa Lozada

Oscar David Arevalo

Parte B

1. Tabla diligenciada

R1	In-orden: izquierda, raíz, derecha	Entrega todos los clientes ordenados de menor a mayor por total de compras.	No. Debe visitar todos los nodos.
R2	In-orden inverso: derecha, raíz, izquierda	Visita primero las claves mayores y permite obtener los 3 clientes con más compras sin ordenar toda la colección.	Sí. Se detiene al encontrar 3 clientes.
R3	In-orden con poda por rango	Entrega los clientes de 25 a 35 años, inclusive, en orden ascendente. Evita visitar ramas que no pueden contener edades dentro del rango.	Sí. Poda ramas fuera del rango.
R5	Búsqueda del mínimo y del máximo	Para el más joven, se sigue por la izquierda; para el de mayor edad, por la derecha.	Sí. Solo recorre una rama hasta llegar al extremo.

Parte C

1. ¿Cuántos clientes se pierden con el código base?

En el arreglo de prueba hay 9 clientes, pero las edades repetidas se descartan porque insertRecursive no realiza ninguna inserción cuando la clave es igual a la del nodo actual.

Las edades son:

34, 28, 45, 28, 52, 31, 23, 39, 34

Las edades distintas son 23, 28, 31, 34, 39, 45, 52. Por tanto:

- Clientes registrados: 9.

- Nodos de arbolEdad con el código original: 7.
- Clientes perdidos del árbol: 2.

Los clientes que se pierden del árbol de edades son Camilo Vélez, que tiene 28 años, y Paola Gil, que tiene 34 años, porque esas claves ya estaban insertadas. Siguen existiendo en el arreglo original y en el árbol de compras si sus claves de compras son diferentes.

3. ¿Qué pasa con getKeyList y clientesEnRango?

getKeyList() debe seguir usando el recorrido in-orden. Con la política de insertar los iguales a la derecha, la lista continúa ordenada de menor a mayor, pero ahora puede contener valores repetidos.

Con los datos del ejercicio, la lista de edades debe ser:

[23, 28, 28, 31, 34, 34, 39, 45, 52]

El método clientesEnRango(25, 35) también conserva los clientes repetidos y devuelve los resultados ordenados por edad. Debe incluir a los dos clientes de 28 años y a los dos de 34 años.

4. ¿Qué ocurre con buscarNodo cuando hay claves repetidas?

El método buscarNodo encuentra el primer nodo con la clave buscada que encuentre desde la raíz. Si hay varios clientes con la misma edad, devuelve solo uno de ellos, no todos.

Una alternativa para recuperar todos sería crear un método buscarTodos(int key) que recorra las ramas pertinentes y guarde en una lista todos los clientes cuya clave coincida con la buscada. Otra opción sería almacenar una lista de clientes en cada nodo, agrupando los que tengan la misma clave.

Parte D

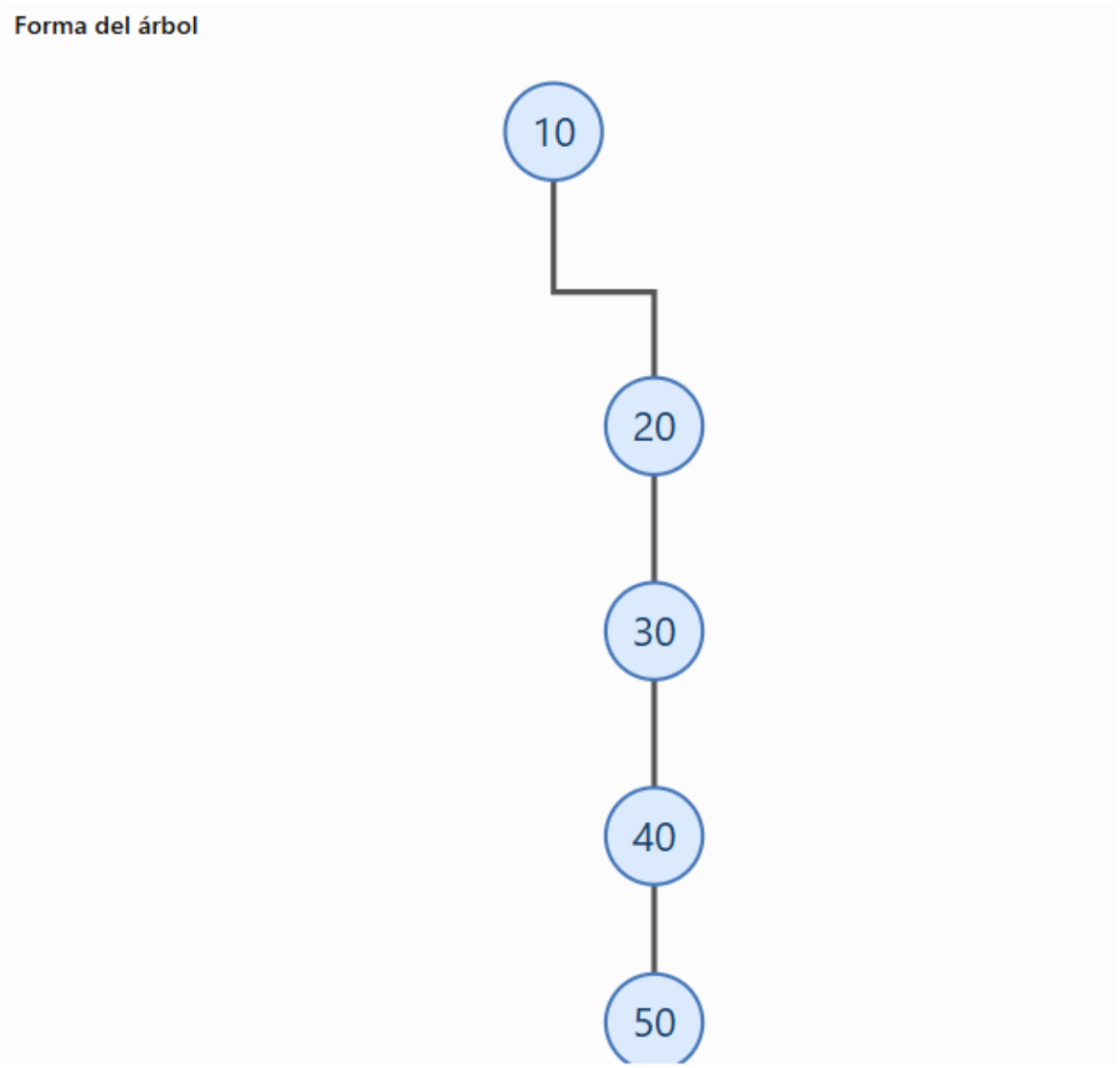
1. Implementar el método altura

Según el enunciado, un árbol vacío tiene altura -1 y un árbol con un solo nodo tiene altura 0. La altura se calcula tomando el máximo entre las alturas de los subárboles izquierdo y derecho, y sumando uno.

2. Construir y dibujar los árboles

Caso A: insertar 10, 20, 30, 40, 50, 60

Al insertar las claves en orden ascendente, cada clave nueva queda a la derecha de la anterior.

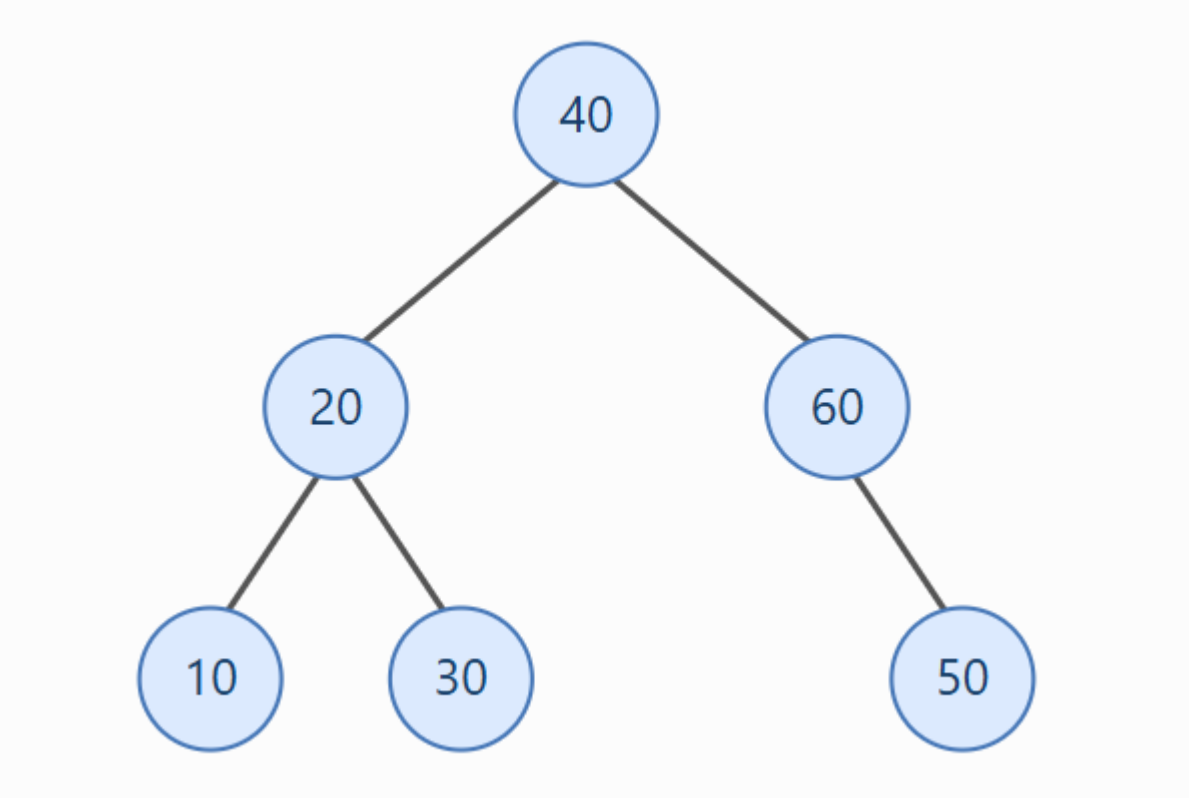
- 

  Número de nodos: 6.
- Altura: 5.

- Complejidad de búsqueda: O(n) en este caso; la búsqueda puede recorrer los seis nodos.

Caso B: insertar 40, 20, 60, 10, 30, 50

El árbol resultante es:



3. ¿Qué complejidad tiene la búsqueda en cada caso?

En el primer árbol, la búsqueda tiene complejidad *O(n)O(n)*O(n) porque los nodos forman una cadena y el árbol está completamente desbalanceado. En el segundo, la forma está balanceada y la búsqueda tiene un comportamiento de orden *O(log⁡n)O(\log n)*O(logn).

En general, la búsqueda en un árbol binario de búsqueda cuesta *O(h)O(h)*O(h), donde *hh*h es la altura. Si el árbol está balanceado, su altura es aproximadamente logarítmica; si está desbalanceado, puede llegar a *n−1n-1*n−1.

¿Qué debería ocurrir con los datos de entrada para evitar el peor caso? ¿Qué estructura resuelve este problema?

Las claves deberían insertarse en un orden que produzca un árbol equilibrado, evitando que queden ordenadas ascendente o descendentemente. Sin embargo, ordenar la entrada por sí solo no garantiza un árbol equilibrado si después se insertan las claves secuencialmente en un BST común. Una solución más confiable es utilizar un árbol AVL o un árbol rojinegro (Red-Black Tree), que mantienen el equilibrio automáticamente y permiten búsquedas, inserciones y eliminaciones en *O(logn)O(\log n)*O(logn).